

Introduction to graph neural networks and anemoi-graphs

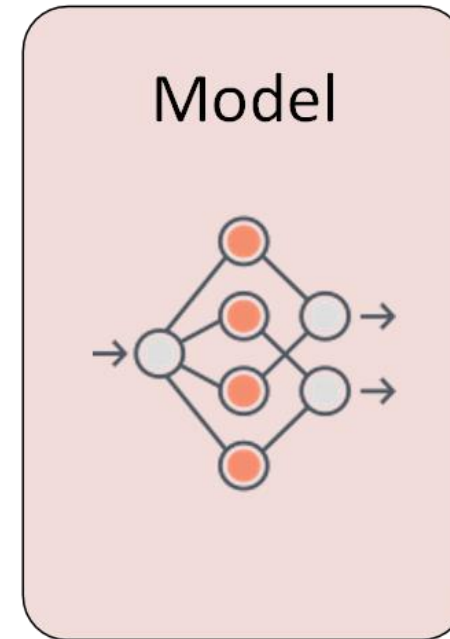
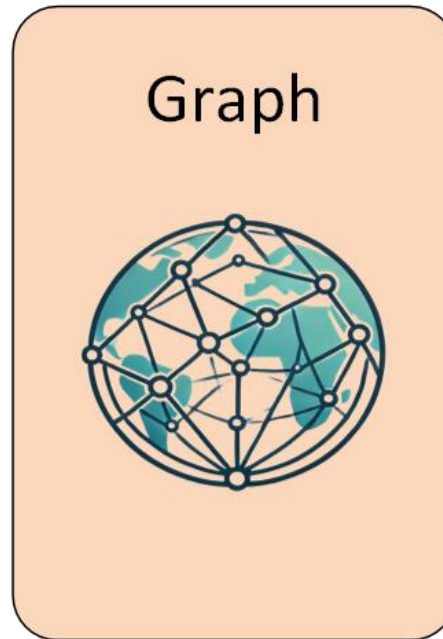
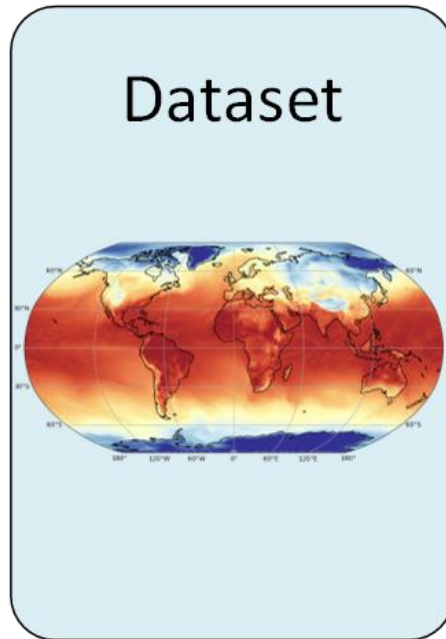
Vera Gahlen

On behalf of many people at ECMWF

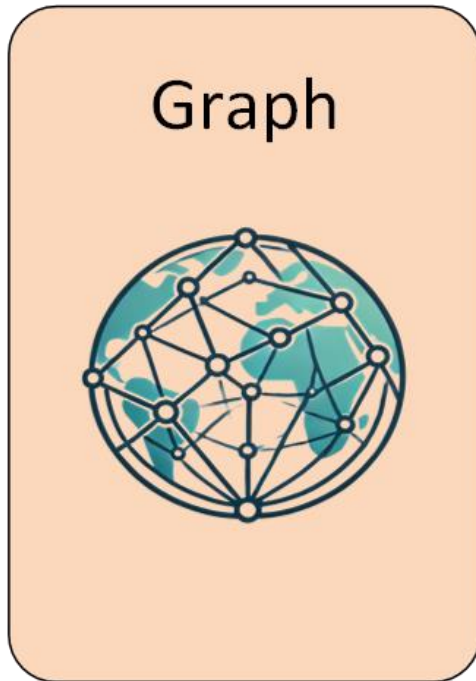
anemoi-graphs

Getting started

To train a data-driven weather forecasting model in **Anemoi**, three main components are needed:



anemoui-graphs



anemoui-graphs.readthedocs.io

In Anemoui, it is represented by a `torch_geometric.data.HeteroData` object, and stored in a .PT file.

- A graph is a structure composed of:
 - **Nodes:** Represent locations in space.
 - **Edges:** Define how information flows between nodes.

```
HeteroData(  
  data={  
    x=[40320, 2], # coordinates in radians (lat in [-pi/2, pi/2], lon in [0, 2pi])  
    node_type='ZarrDatasetNodes',  
    area_weight=[40320, 1],  
  },  
  hidden={  
    x=[10242, 2], # coordinates in radians (lat in [-pi/2, pi/2], lon in [0, 2pi])  
    node_type='TriNodes',  
    area_weight=[10242, 1],  
  },  
  (data, to, hidden)={  
    edge_index=[2, 62980],  
    edge_type='CutOffEdges',  
    edge_length=[62980, 1],  
    edge_dirs=[62980, 2],  
  },  
  (hidden, to, hidden)={  
    edge_index=[2, 81900],  
    edge_type='MultiScaleEdges',  
    edge_length=[81900, 1],  
    edge_dirs=[81900, 2],  
  },  
  (hidden, to, data)={  
    edge_index=[2, 120960],  
    edge_type='KNNEdges',  
    edge_length=[120960, 1],  
    edge_dirs=[120960, 2],  
  }  
)
```

Console log of a graph created with anemoui-graphs.

anemoi-graphs

Tools for creating graphs used in data-driven, deep learning weather forecasting models.

- **High-Level Interface:**
 - YAML recipe file to define graph configuration.
- **Node Definition:**
 - Based on dataset coordinates (e.g., Zarr, NPZ, TXT, ...) or algorithmic methods (e.g., triangular refined icosahedron).
- **Edge Definition:**
 - Methods include cut-off radius or K nearest-neighbors.
- **Attributes:**
 - Supports node and edge attributes (weights, lengths, directions).

anemoi-graphs.readthedocs.io

The screenshot shows the documentation page for the `anemoi-graphs` package. The left sidebar contains a navigation menu with sections: INTRODUCTION, BUILDING GRAPHS, RECIPE EXAMPLES, MODULES, and DEVELOPING ANEMOI GRAPHS. The main content area is titled "Command line tool" and explains that a command line tool is installed with the package. It provides instructions on how to use the `anemoi-graphs` command with various options like `--help`, `create`, and `describe`. It also shows the output of the `describe` command, which generates a text summary of the graph, including node and edge statistics.

Command line tool

When you install the `anemoi-graphs` package, a command line tool will also be installed called `anemoi-graphs`, which can be used to build graphs based on YAML recipe files, and inspect existing graphs.

The tool can provide help with the `--help` options:

```
$ anemoi-graphs --help
```

To create a graph, use the `create` command:

```
$ anemoi-graphs create recipe.yaml graph.pt
```

The `recipe.yaml` file consists of high-level specifications for generating the graphs at each layer. An example of a simple recipe file is given in the [following section](#).

The `create` command will read the specifications in the `recipe.yaml` file, and write to a PyTorch `.pt` file.

To describe an existing graph stored as a `.pt` file, use the `describe` command:

```
$ anemoi-graphs describe graph.pt
```

This will generate a text summary of the graph, including the number of nodes and edges at each layer, the geographic boundaries, and statistics about the edge lengths:

Nodes name	Num. nodes	Attribute dim	Min. latitude	Max. latitude	Min. longitude	Max. longitude
data	10,840	4	-3.135	3.140	0.02	0.02
hidden	6,200	4	-3.141	3.137	0.01	0.01

Source	Destination	Num. edges	Attribute dim	Min. length	Max. length	Mean
data	hidden	13508	1	0.3116	25.79	11.
hidden	data	40910	1	0.2397	21.851	12.

Documentation of anemoi-graphs package.

Command line tool

- Create a new graph:

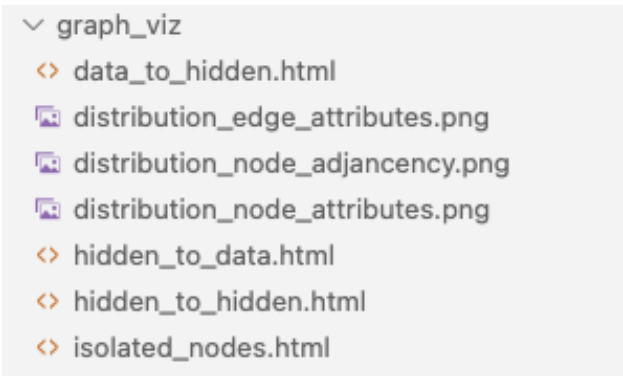
```
>>> anemoi-graphs create recipe.yaml graph.pt
```

- Describe an existing graph:

```
>>> anemoi-graphs describe graph.pt
```

- Inspect visually an existing graph:

```
>>> anemoi-graphs inspect graph.pt graph_viz/
```



Local files generated to inspect graphs.

Path

: graph.pt

Format version:

0.0.1

Size

: 3.1 MiB (3,283,650)

Nodes name	Num. nodes	Attribute dim	Min. latitude	Max. latitude	Min. longitude
data	10,840	4	-3.135	3.140	0.02
hidden	6,200	4	-3.141	3.137	0.01

Source	Destination	Num. edges	Attribute dim	Min. length	Max. length	Mean
data	hidden	13508	1	0.3116	25.79	11
hidden	data	40910	1	0.2397	21.851	12

Graph ready, last update 17 seconds ago.

Statistics ready.

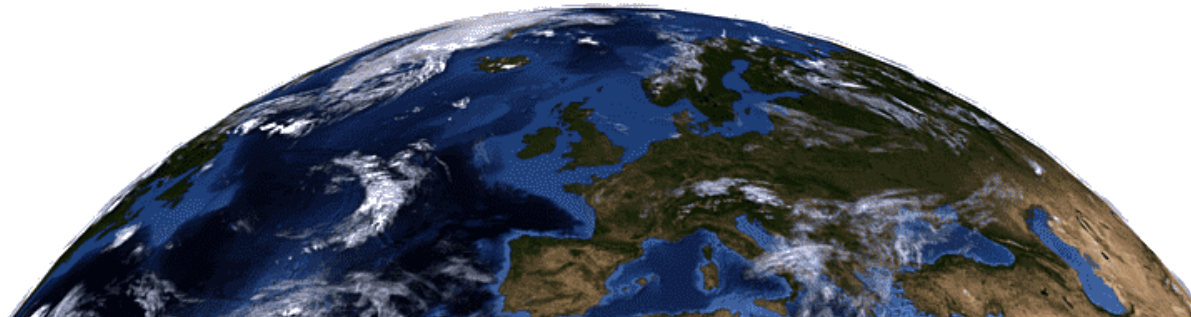
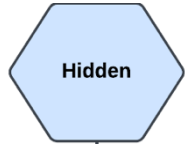
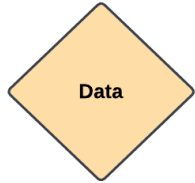
Console log when describing/inspecting a graph with anemoi-graphs.

Note: The inspection tools provided are designed for testing different graph configuration but it is not recommended for high-resolution graphs with a high number of nodes/edges.

Encoder – Processor – Decoder

anemoi-training uses a specific **encoder-processor-decoder** graph structure:

- **Data Nodes:** Represent the locations of input/output data points (e.g., temperature, wind speed).
- **Hidden Nodes:**
 - Operate at a lower spatial resolution than the input/output nodes. ($\sim 1/2$)
 - Allow for feature extraction and dimensionality reduction, enabling the model to capture large-scale patterns.



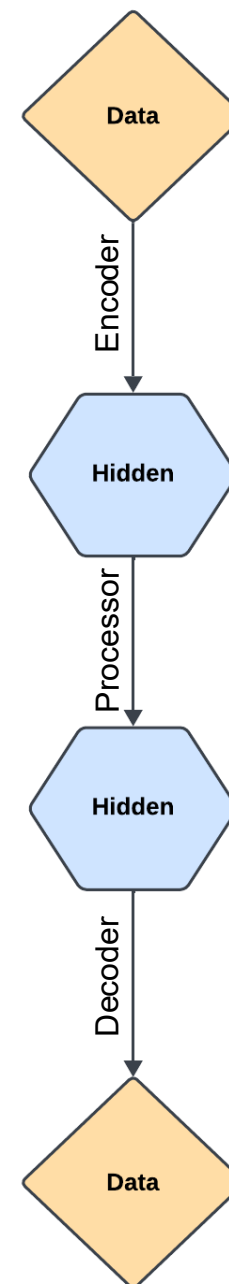
Note: For weather forecasting, the input and output nodes often correspond to the same physical locations. In other use cases (e.g., downscaling), input and output nodes may differ.

Graph recipe

recipe.yaml

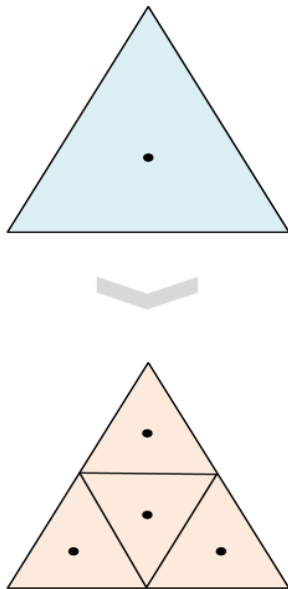
```
nodes:
  data:
    node_builder:
      _target_: anemoi.graphs.nodes.ZarrDatasetNodes
      dataset: my_zarr_dataset
  hidden:
    node_builder:
      _target_: anemoi.graphs.nodes.TriNodes
      resolution: 5 # num of refinements

edges:
  # Encoder configuration
  - source_name: data
    target_name: hidden
    edge_builders:
      - _target_: anemoi.graphs.edges.CutOffEdges
        cutoff_factor: 0.6
  # Processor configuration
  - source_name: hidden
    target_name: hidden
    edge_builders:
      - _target_: anemoi.graphs.edges.MultiScaleEdges
        x_hops: 1
  # Decoder configuration
  - source_name: hidden
    target_name: data
    edge_builders:
      - _target_: anemoi.graphs.edges.KNNEdges
        num_nearest_neighbours: 3
```

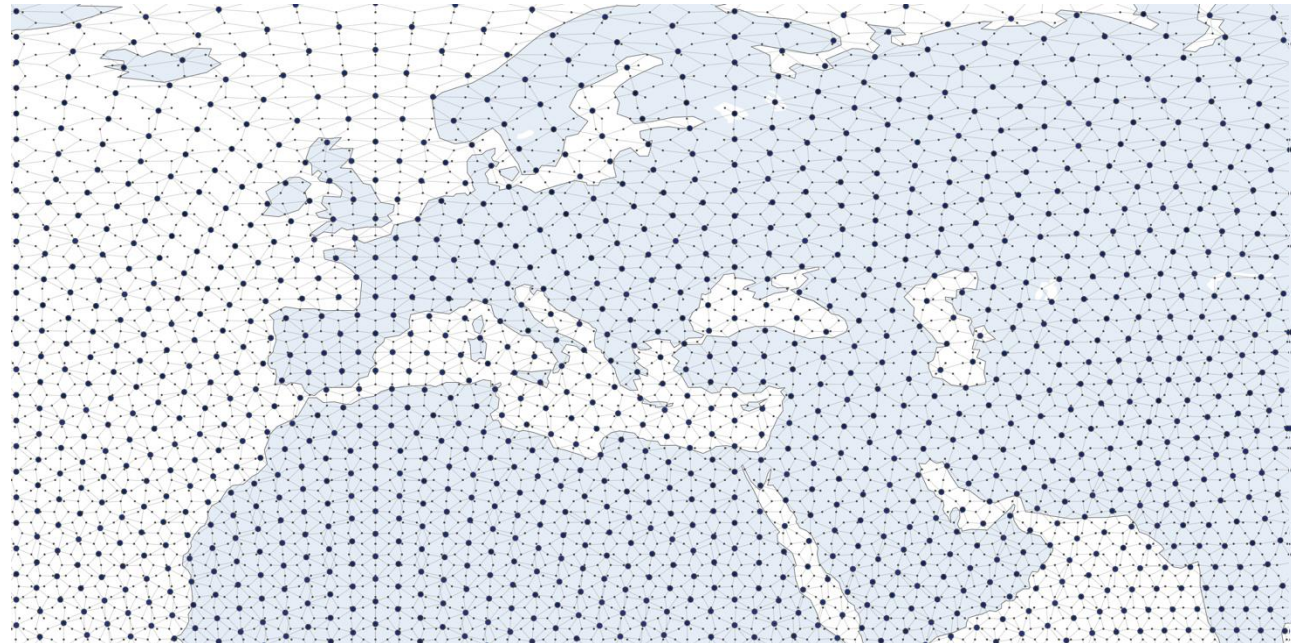


Triangular refined mesh

- It starts with an icosahedron (20 faces) projected to a sphere.
- Each refinement splits each face into 4 faces.



```
nodes:  
  hidden:  
    node_builder:  
      _target_: anemoi.graphs.nodes.TriNodes  
      resolution: 5 # num. of refinements
```

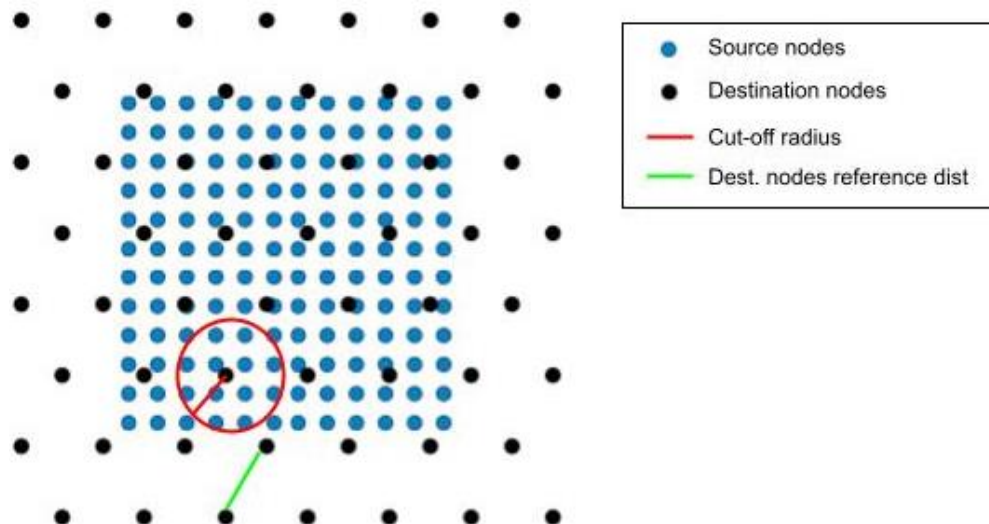


Encoder - CutOffEdges

For each target node, it connects all nodes within a given radius.

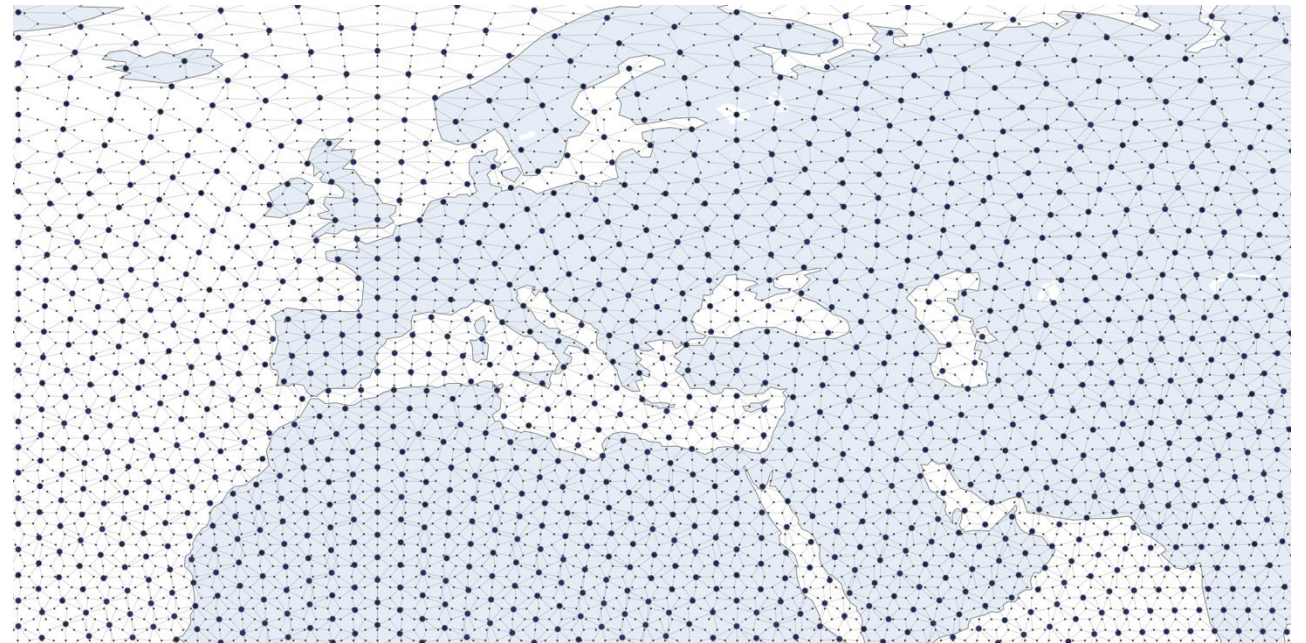
Why is this important?

- You want to maximise the information flowing to the hidden nodes.
- You want to minimise the number of edges (efficiency)



edges:

- source_name: **data**
- target_name: **hidden**
- edge_builders:
 - _target_: anemoi.graphs.edges.CutOffEdges
 - cutoff_factor: 0.6

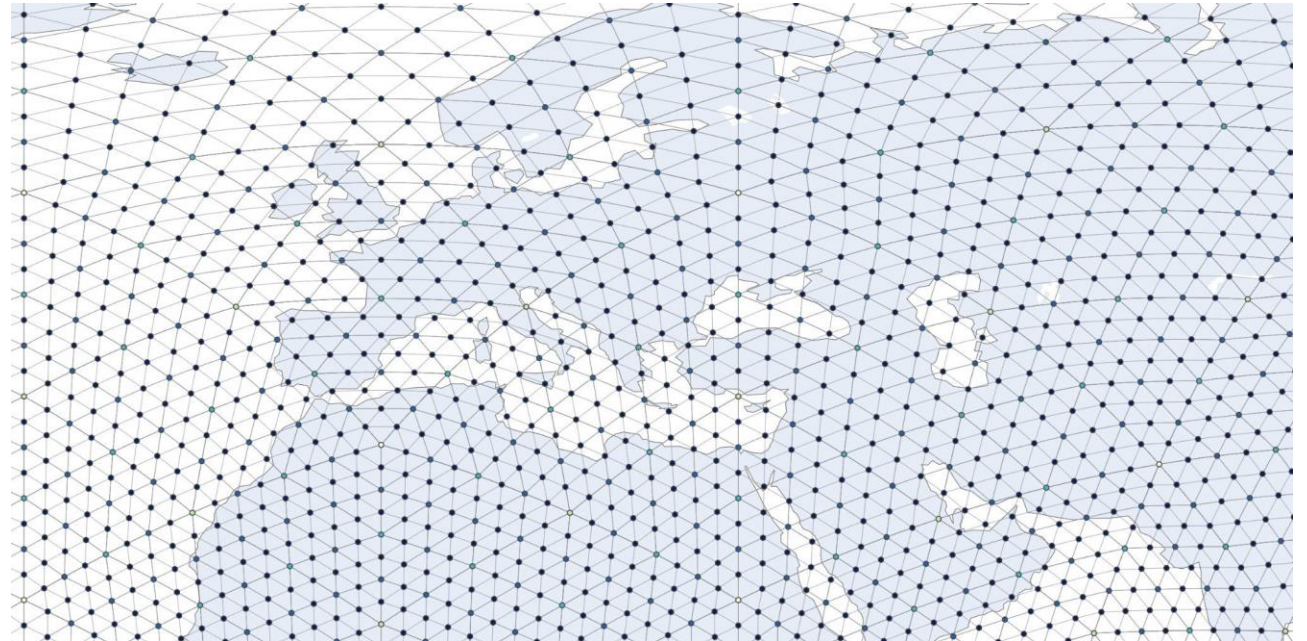
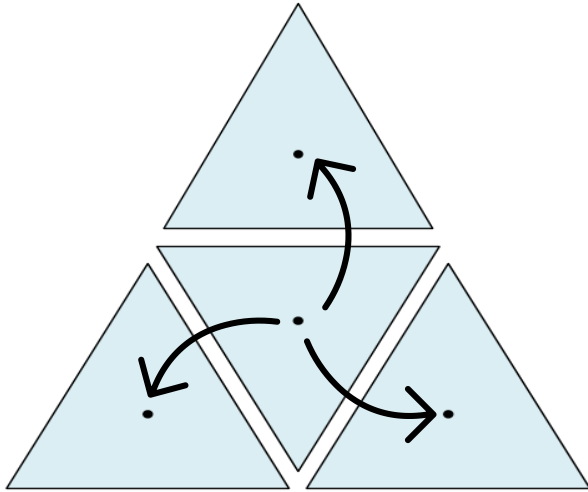


Processor - MultiScaleEdges

- The processor connections define how far the information flows.
 - Connections are created for each level of refinement.

edges:

- source_name: **hidden**
- target_name: **hidden**
- edge_builders:
 - _target_: anemoi.graphs.edges.**MultiScaleEdges**
 - x_hops: 1



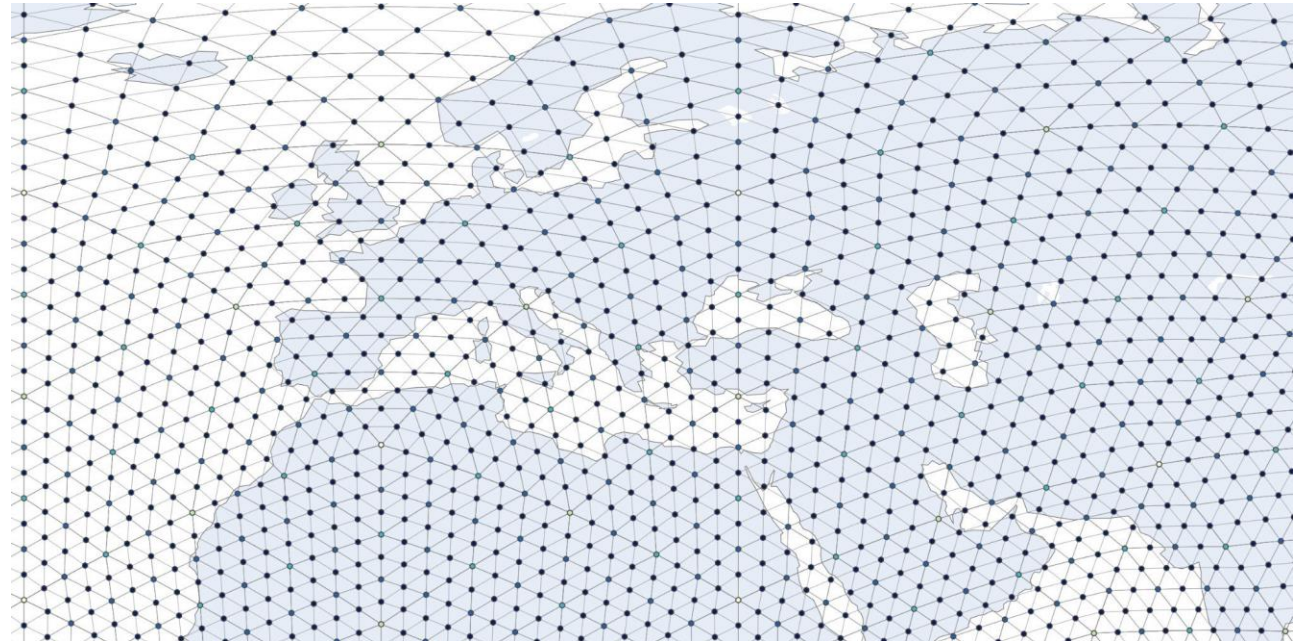
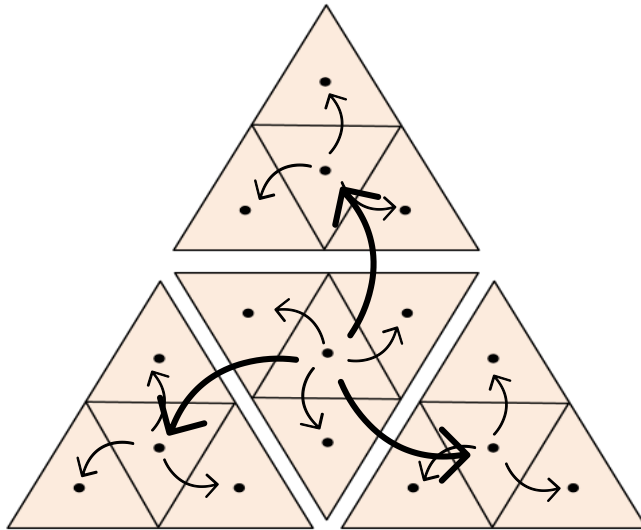
Processor - MultiScaleEdges

- The processor connections define how far the information flows.

- Connections are created for each level of refinement.

edges:

- source_name: **hidden**
- target_name: **hidden**
- edge_builders:
 - _target_: anemoi.graphs.edges.**MultiScaleEdges**
 - x_hops: 1

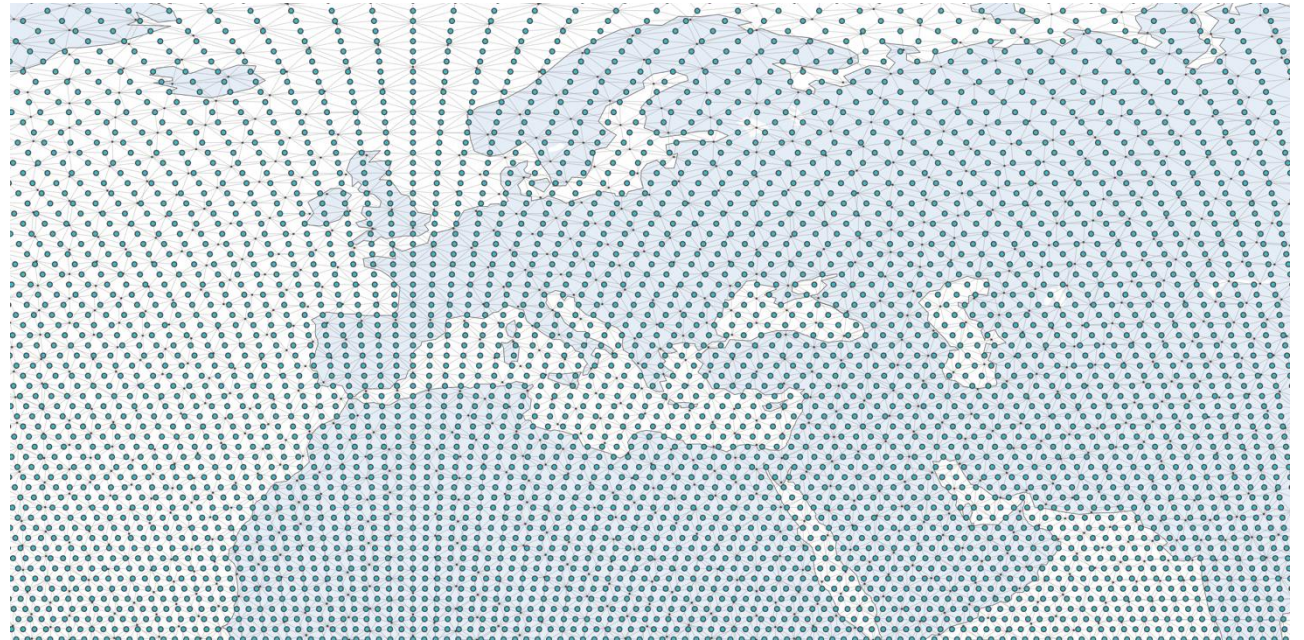


Decoder - KNNEdges

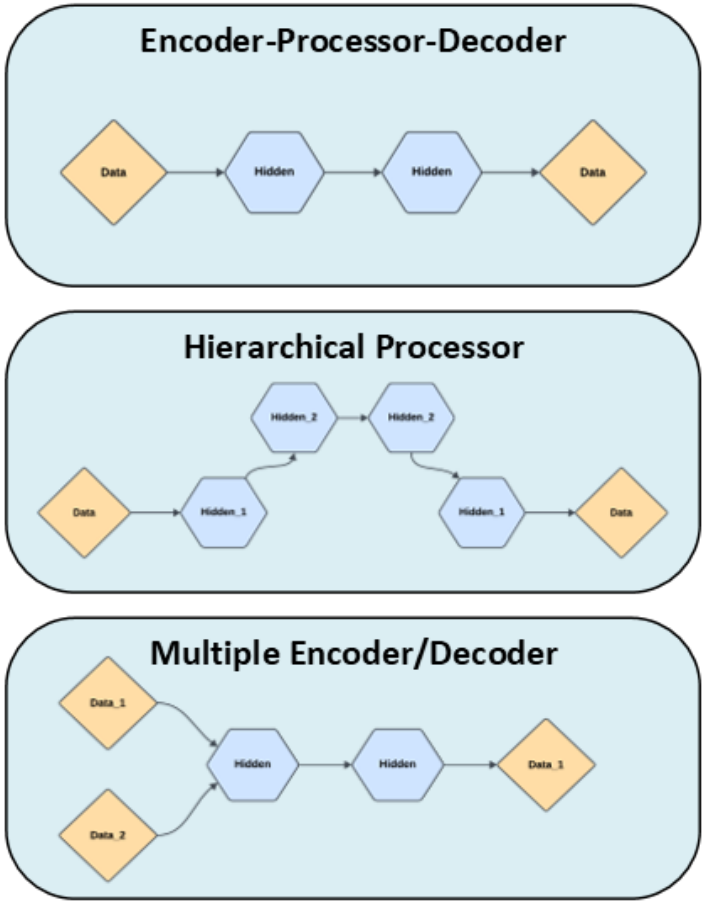
- For each target node, it connects the N nearest neighbours.

```
edges:  
- source_name: hidden  
  target_name: data  
  edge_builders:  
    - _target_: anemoi.graphs.edges.KNNEdges  
      num_nearest_neighbours: 3
```

NOTE: The decoder is the part of the graph with more edges.



Graph configurations



The screenshot shows the GitHub interface for the 'anemoi-core' repository. The 'Files' tab is active, displaying the directory structure. The 'training' directory is expanded, showing a list of files and folders. A commit message is visible at the top of the file list: 'JPXKQX chore(training): Add default config files for 2 and 3 level hierarchi...'. Below the file list, the 'README.md' file is open, showing the title 'anemoi-training'.

Name	Last commit message	Last commit date
..		
docs	Merge commit '38b75fadd5fcd935547e8239180a92801...	last month
src	chore(training): Add default config files for 2 and 3 level hi...	19 hours ago
tests	build: fix isort and pre-commits	last month
.gitattributes	Add 'training/' from commit 'c99069ee00147e889c947f71...	last month
.gitignore	Add 'training/' from commit 'c99069ee00147e889c947f71...	last month
.readthedocs.yaml	docs: point RTD to right subfolder	last month
CHANGELOG.md	chore(training): Add default config files for 2 and 3 level hi...	19 hours ago
CONTRIBUTORS.md	Add 'training/' from commit 'c99069ee00147e889c947f71...	last month
LICENSE	Add 'training/' from commit 'c99069ee00147e889c947f71...	last month
README.md	docs: Tidy for core	last month
pyproject.toml	bump anemoi-datasets required version to 0.5.13 (#74)	yesterday
pytest.ini	Add 'training/' from commit 'c99069ee00147e889c947f71...	last month

Example [configuration files](#) for different use cases.

Use cases

Stretched graphs



Limited area graphs



Some user may focus in specific regions of interest. In this case, there are several options:

- **Stretched graphs**
 - Increases resolution over a region of interest while maintaining a coarser grid elsewhere.
- **Limited area graphs**
 - Nodes are restricted to a specific region.
 - A coarser dataset can be used as *boundary forcing*.

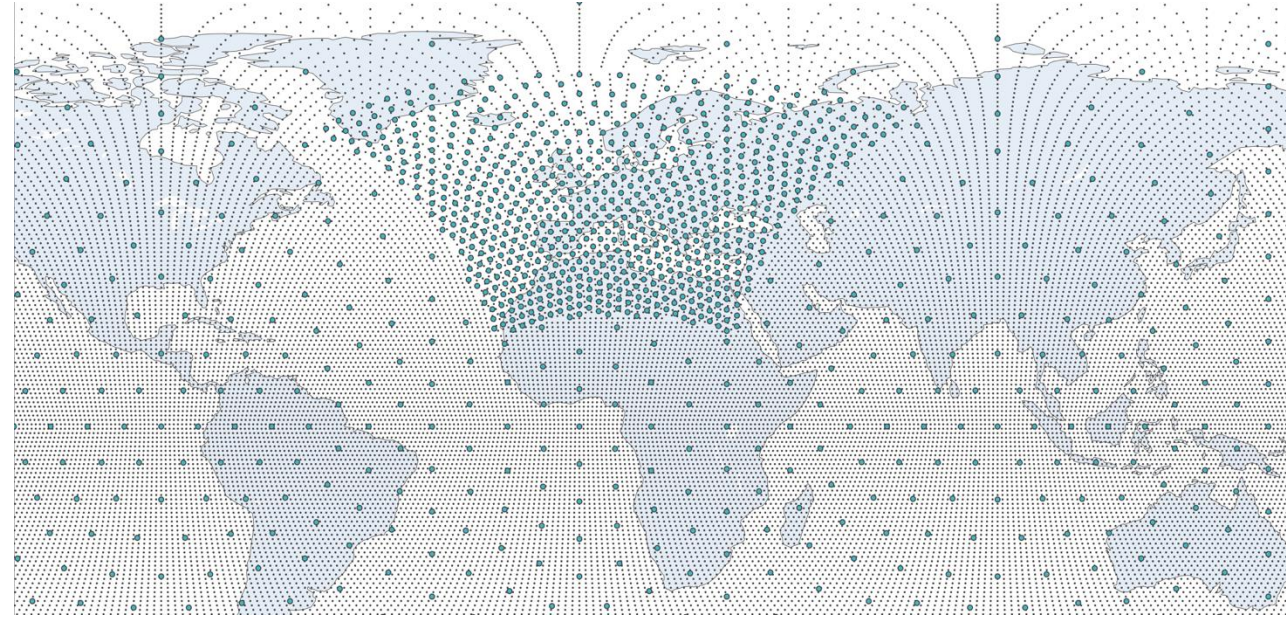
Regional modeling

```
nodes:
  data:
    node_builder:
      _target_: anemoi.graphs.nodes.ZarrDatasetNodes
    dataset:
      cutout:
        - my_local_zarr_dataset
        - my_global_zarr_dataset
      adjust: all
    attributes:
      area_weight:
        _target_: anemoi.graphs.nodes.attributes.SphericalAreaWeights
      norm: unit-max
    cutout_mask:
      _target_: anemoi.graphs.nodes.attributes.CutOutMask
```

Graph recipe file containing the data nodes configuration.

```
hidden:
  node_builder:
    _target_: anemoi.graphs.nodes.LimitedAreaTriNodes
  resolution: 5
  reference_node_name: data
  node_attr_name: cutout_mask
```

Graph recipe file containing the nodes configuration for LAM.



Data and hidden nodes for a stretched graph.

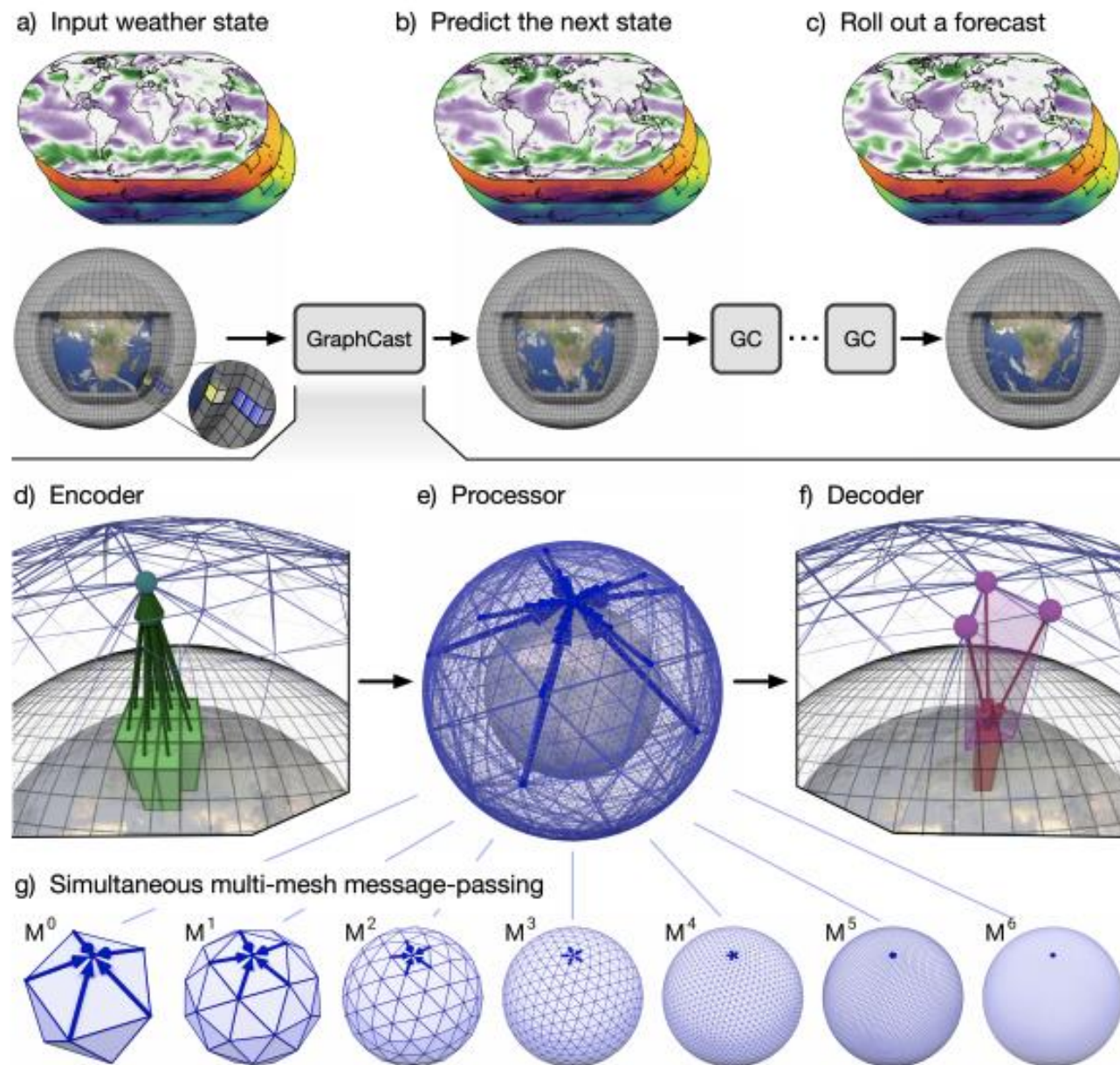
```
hidden:
  node_builder:
    _target_: anemoi.graphs.nodes.StretchedTriNodes
  lam_resolution: 5
  global_resolution: 3
  reference_node_name: data
  node_attr_name: cutout_mask
```

Graph recipe file containing the nodes configuration for stretched graphs.

Wrap up

- *anemoi-graphs* is used by *anemoi-training* to create the graphs on the fly.
- *anemoi-training* contains recipes for all use cases
- *anemoi-graphs* has a command line tool to create and inspect graphs
- *anemoi-graphs* is designed to be extended with new node, edge and attribute builders.
- *anemoi-graphs* supports different use cases:
 - Global graphs
 - Limited area graphs
 - Stretched graphs
- More complex setups (multi-encoder/decoder, dynamic graphs, ...) are not supported outside *anemoi-graphs*.
- Graph configuration plays a key role in the flow of information and the efficiency of the model.

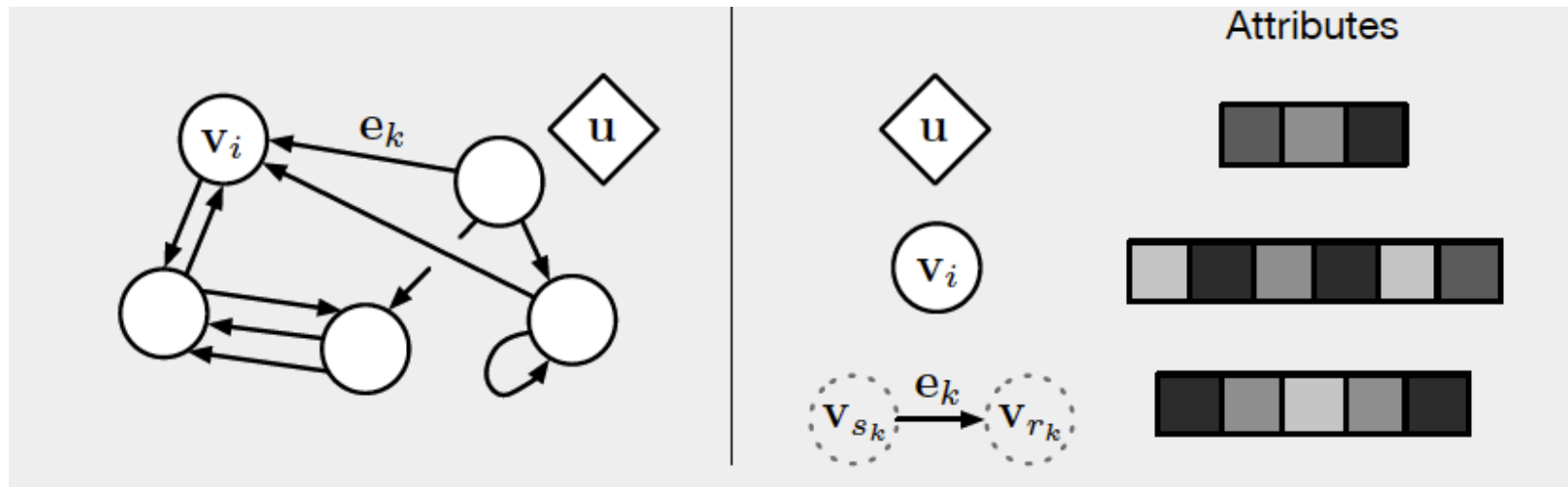
Graph Neural Networks



Refresher on graphs

We define a *graph* as the pair $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ of vertices $v \in \mathcal{V}$ and edges $e_{ij} = (v_i, v_j) \in \mathcal{E}$ with $\mathcal{E} \subseteq V \times V$. The graph connectivity is encoded as an *adjacency matrix* $A \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$, with

$$a_{ij} = \begin{cases} 1, & e_{ij} \in \mathcal{E} \\ 0, & e_{ij} \notin \mathcal{E} \end{cases}$$



Graph neural networks

We endow the graph nodes with features - namely for each $u \in \mathcal{V}$ we define a *node feature tensor* $\mathbf{x} \in \mathbb{R}^k$. This defines a matrix $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times k}$:

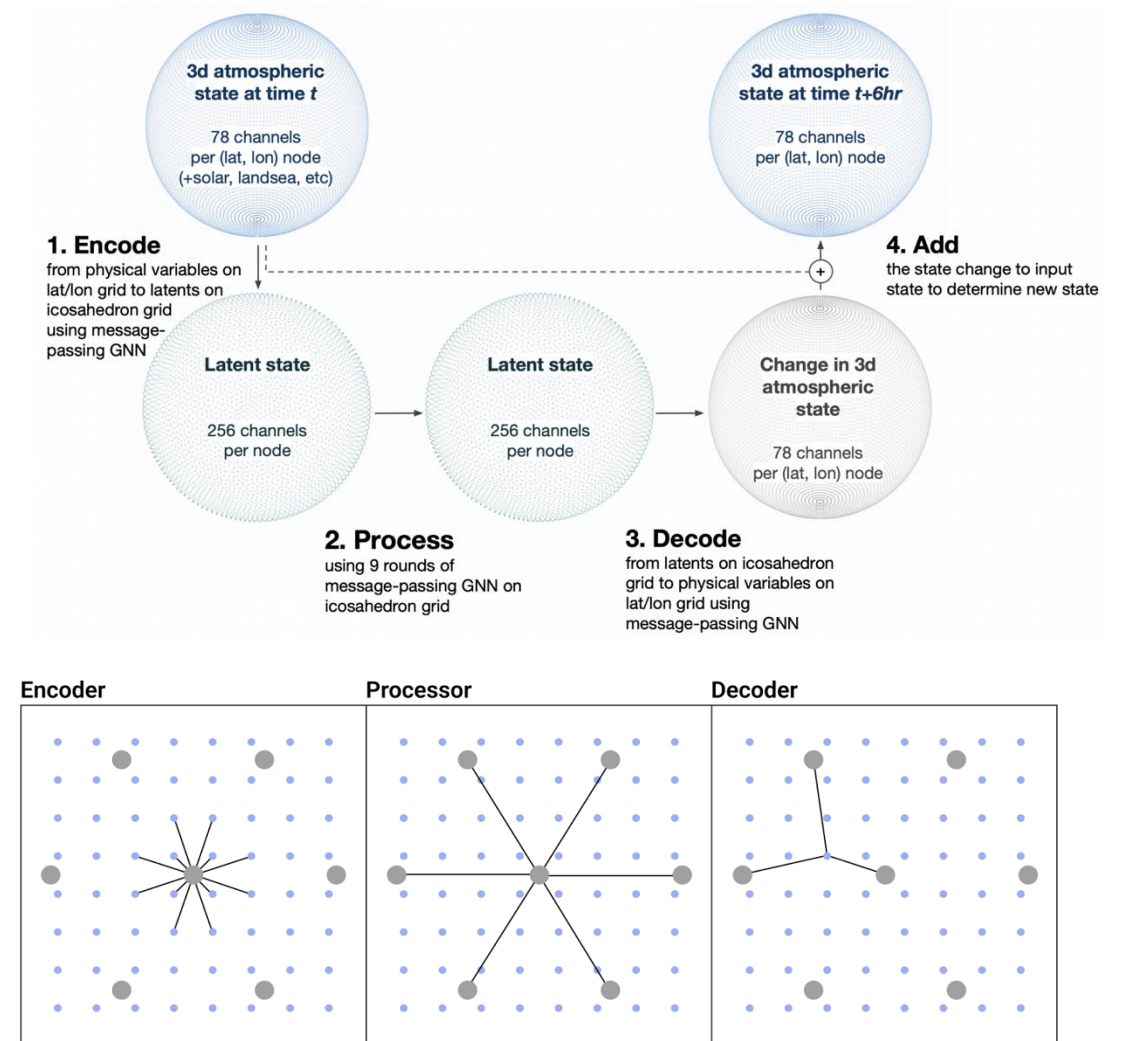
$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{|\mathcal{V}|}]$$

We also define *edge features* $\mathbf{x}_{uv} \in \mathbb{R}^l$ and global *graph features* $\mathbf{x}_{\mathcal{G}} \in \mathbb{R}^m$.

GNNs are neural networks built to operate on this “graph data”.

Graph neural networks

- Information able to pass between connected “nodes”
- Can specify features along graph edges (e.g. distance, direction, etc.)
- Originated from social network modelling and suggestion algorithms
- Natural choice for atmospheric modelling
 - Close analogies to physical models passing information between neighboring grid-cells



Great intro about GNNs ->
<https://distill.pub/2021/gnn-intro/>

What inductive biases should a GNN have?

Locality

We want the GNN signal to be stable under small domain deformations.

Standard deep NNs (e.g., CNNs) build large-scale ops from small-scale building blocks (e.g. 3x3 convolutions).

GNN layers should operate locally, too (in neighborhoods).

We can extract neighborhood features and define local functions ϕ (MLPs) operating on them:

$$\mathbf{X}_{\mathcal{N}_i} = \{ \{ \mathbf{x}_j : j \in \mathcal{N}_i \} \}$$

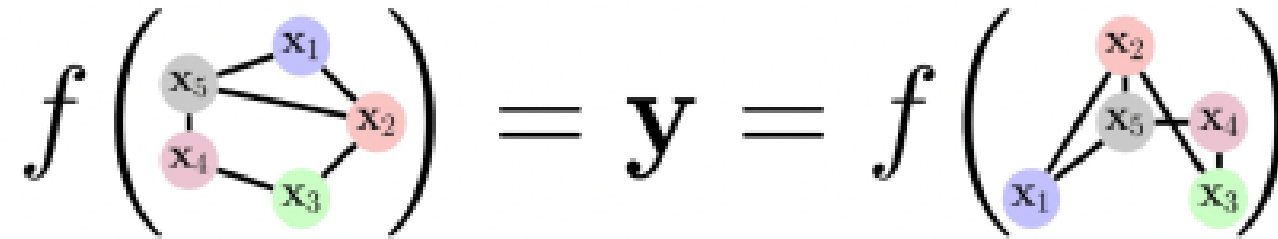
$$\phi(\mathbf{x}_i, \mathbf{X}_{\mathcal{N}_i}).$$

Permutation invariance and equivariance

The specific ordering of nodes / edges should not matter!

Invariance

$$f(PX, PAP^T) = f(X, A)$$



The diagram shows two graphs enclosed in large parentheses, separated by an equals sign and a bold y . The left graph has nodes x_1 (blue), x_2 (red), x_3 (green), x_4 (pink), and x_5 (grey). Edges connect x_1 to x_2 and x_5 ; x_2 to x_3 and x_4 ; x_3 to x_4 ; and x_4 to x_5 . The right graph has nodes x_1 (blue), x_2 (red), x_3 (green), x_4 (pink), and x_5 (grey). Edges connect x_1 to x_2 and x_5 ; x_2 to x_3 and x_4 ; x_3 to x_4 ; and x_4 to x_5 . This represents a permutation of the nodes.

$$f \left(\begin{array}{c} x_1 \\ x_5 \\ x_4 \\ x_3 \end{array} \right) = \mathbf{y} = f \left(\begin{array}{c} x_2 \\ x_5 \\ x_4 \\ x_3 \end{array} \right)$$

Examples: max, sum, min, avg

$\oplus$ = any permutation-invariant aggregation op acting on one or more graph nodes / edges

Permutation equivariance

What if we wanted to distinguish between outputs at different nodes?

A permutation-invariant aggregator would not allow us to do that ☹

Instead, we may use functions that don't change the node ordering.

That is, if we permute nodes using a *permutation matrix* P , it doesn't matter if we do it before or after! ☺

$$\mathbf{F}(\mathbf{X}, \mathbf{A}) = \begin{bmatrix} - & \phi(\mathbf{x}_1, \mathbf{X}_{\mathcal{N}_1}) & - \\ - & \phi(\mathbf{x}_2, \mathbf{X}_{\mathcal{N}_2}) & - \\ & \vdots & \\ - & \phi(\mathbf{x}_n, \mathbf{X}_{\mathcal{N}_n}) & - \end{bmatrix}$$

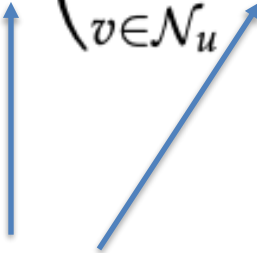
$$F(PX, PAP^T) = PF(X, A).$$

We then stack multiple equivariant GNN layers to build large-scale operators:

$$\mathbf{F}(\mathbf{X}, \mathbf{A}) := \phi \left(\bigoplus_{v \in \mathcal{N}_u} \psi(\mathbf{x}_u, \mathbf{x}_v, \mathbf{x}_{uv}) \right)$$

$\bigoplus$ = any permutation-invariant aggregation op acting on one or more graph nodes / edges

We've just defined a GNN layer!

$$\mathbf{F}(\mathbf{X}, \mathbf{A}) := \phi \left(\bigoplus_{v \in \mathcal{N}_u} \psi(\mathbf{x}_u, \mathbf{x}_v, \mathbf{x}_{uv}) \right)$$


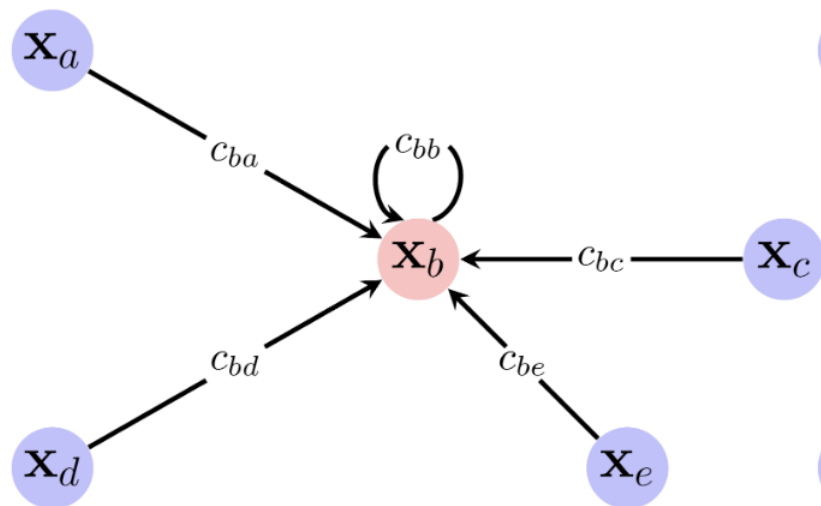
Trainable, shared MLPs

GNN layers are defined by the shared application of local, differentiable and permutation invariant MLPs

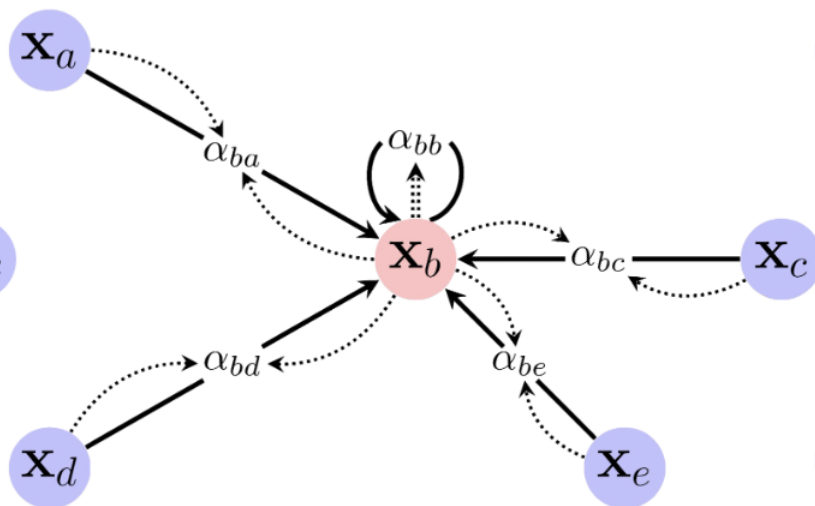
The per-edge and per-node functions ϕ and ψ are reused across all edges and all nodes, respectively. This means a GNN can operate on graphs of different sizes ($|\mathcal{V}|$ or $|\mathcal{E}|$) and shapes ($\mathbf{A}$). This is crucial if your graph is dynamic, e.g. it varies with time.

Flavors of GNNs

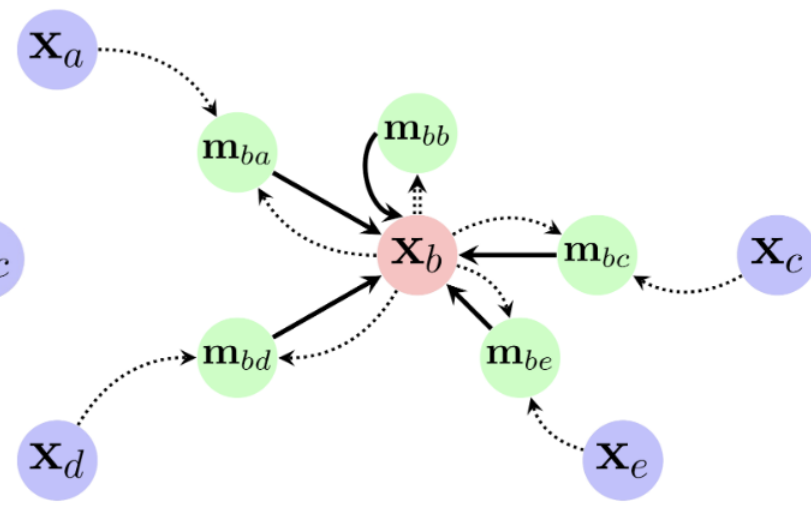
$$h_i = \phi(x_i, \bigoplus_{j \in \mathcal{N}_i} \alpha(x_i, x_j) \psi(x_j))$$



Convolutional



Attentional



Message-passing

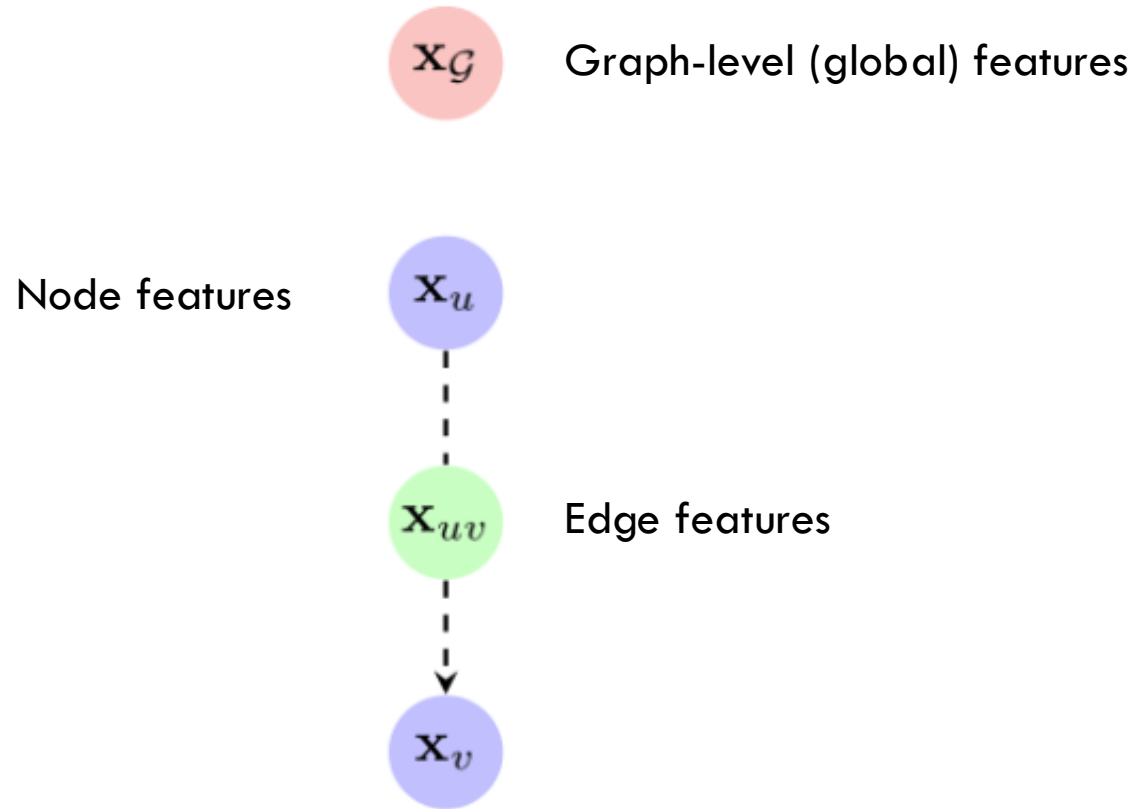
$$h_i = \phi(x_i, \bigoplus_{j \in \mathcal{N}_i} c_{ij} \psi(x_j))$$

$$h_i = \phi(x_i, \bigoplus_{j \in \mathcal{N}_i} \psi(x_i, x_j, e_{ij}))$$

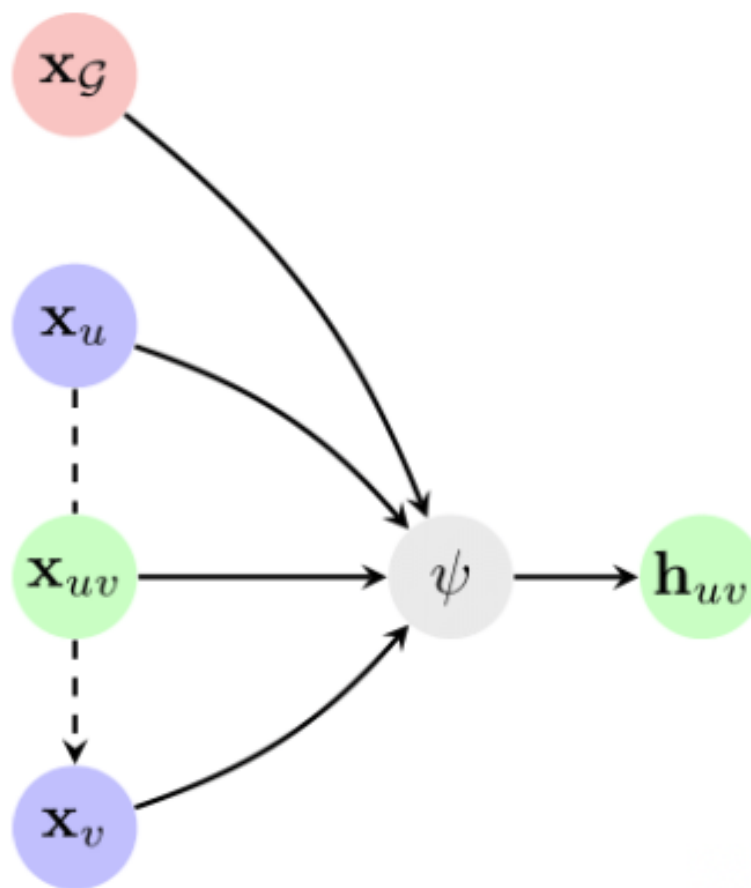
$$\mathbf{m}_{ij} := \psi(x_i, x_j, e_{ij})$$

Message-passing GNNs

Features (= information associated with elements of our graph)

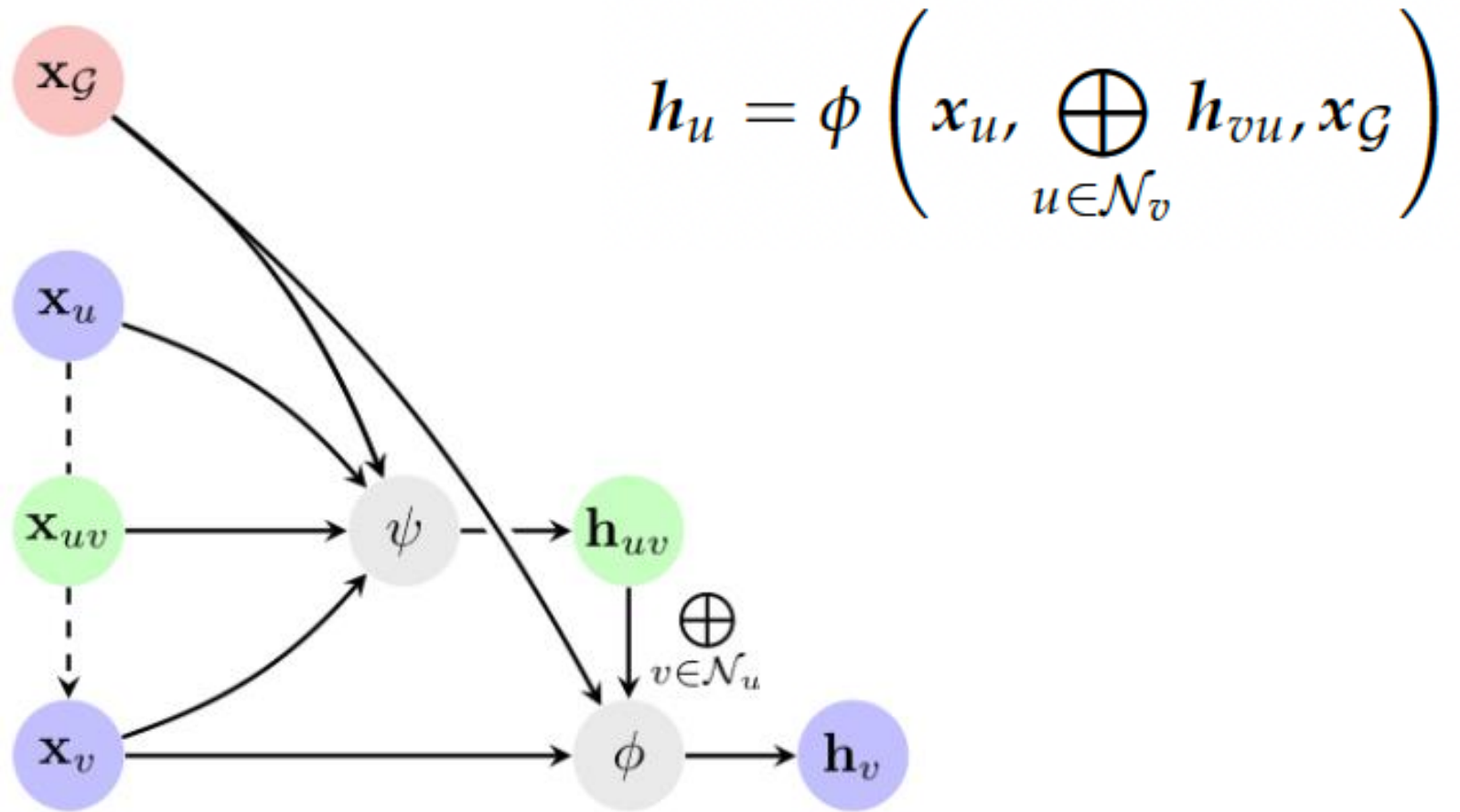


Step 1: Edge updates

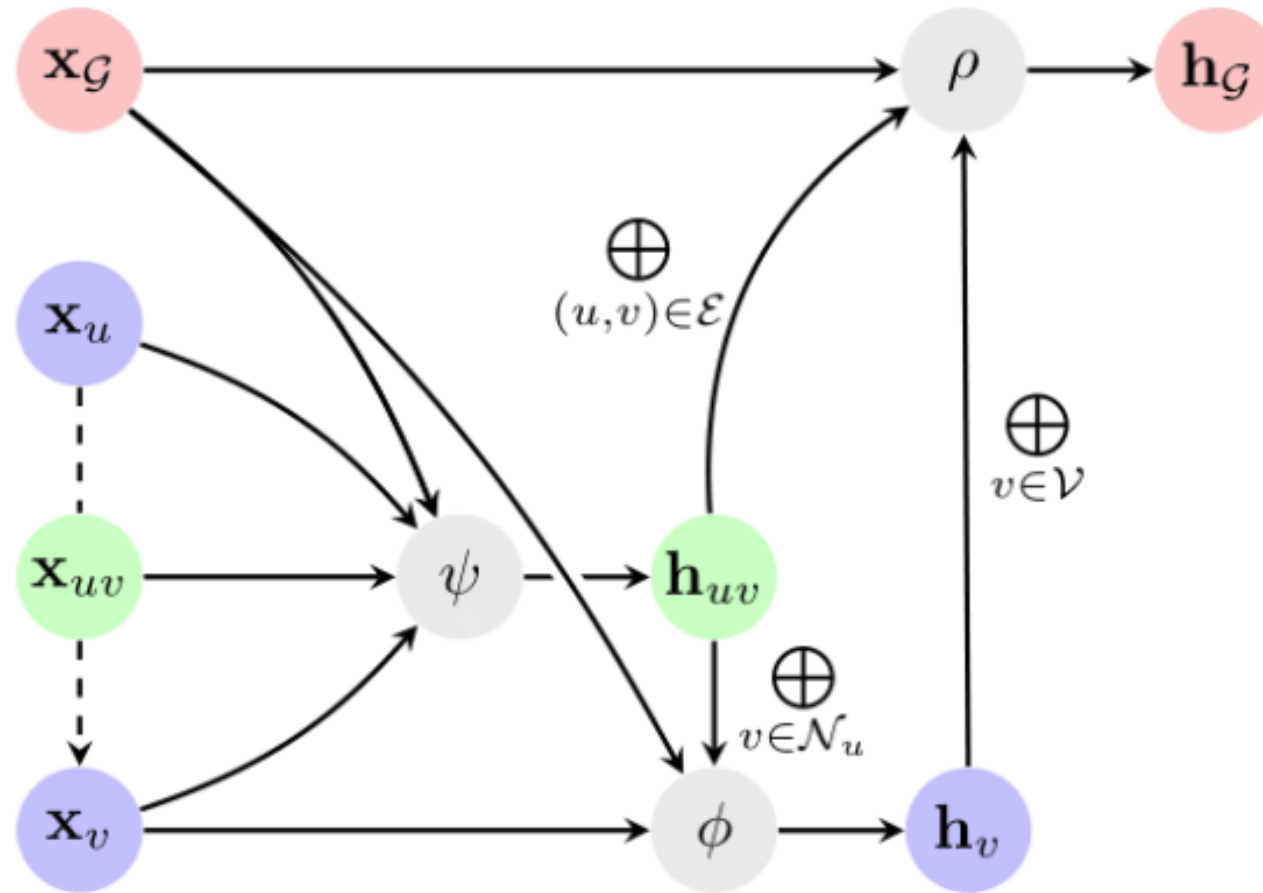


$$h_{uv} = \psi(x_u, x_v, x_{uv}, x_G)$$

Step 2: Node updates



Step 3: Graph feature updates



$$h_G = \rho \left(\bigoplus_{u \in \mathcal{V}} h_u, \bigoplus_{(u,v) \in \mathcal{E}} h_{uv}, x_G \right)$$

The message-passing algorithm

Input: Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ with $\{x_{\mathcal{G}}, h_u, h_{uv}\}$.

for each edge e_{uv} **do**

 Gather sender and receiver nodes x_u, x_v

 Update edge $h_{uv} \leftarrow \psi(x_u, x_v, x_{uv}, x_{\mathcal{G}})$

end for

for each node u **do**

 Aggregate all incoming edges to u : $h_u^* := \bigoplus_{v, (v,u) \in \mathcal{E}} h_{vu}$

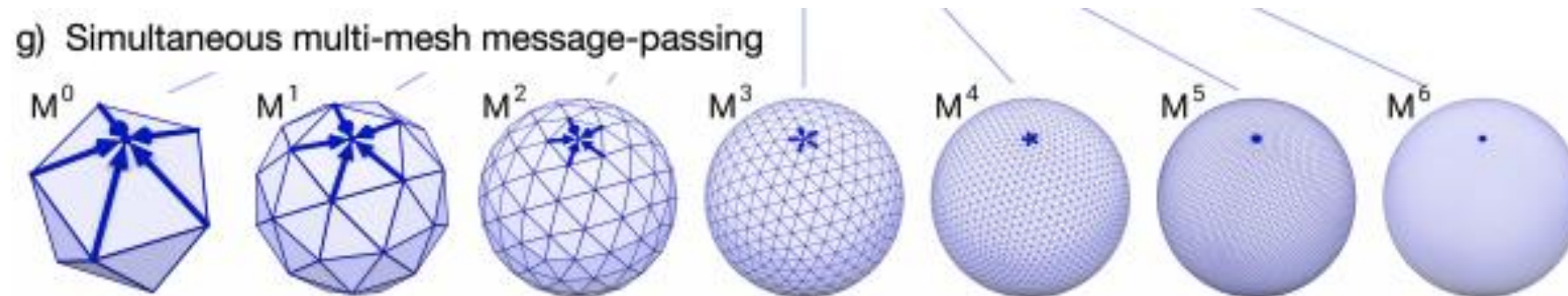
 Compute node-wise features $h_u \leftarrow \phi(x_u, h_u^*, x_{\mathcal{G}})$

end for

Aggregate all edges and nodes $u^* := \bigoplus_{u \in \mathcal{V}} h_u, \mathbf{e}^* := \bigoplus_{(u,v) \in \mathcal{E}} h_{uv}$

Compute global features $h_{\mathcal{G}} \leftarrow \rho(x_{\mathcal{G}}, u^*, \mathbf{e}^*)$

Output: Graph $\mathcal{G}$ with new $\{x_{\mathcal{G}}, h_u, h_{uv}\}$.

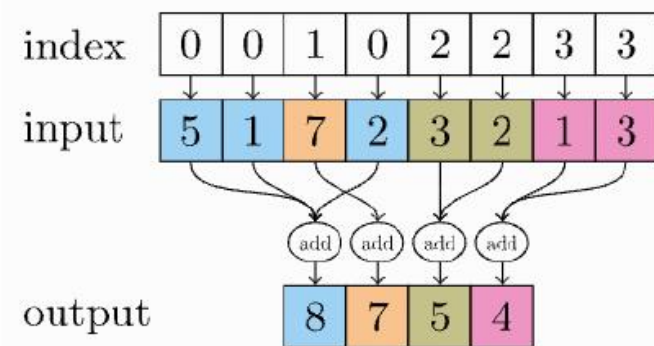


The multi-mesh allows information to propagate faster, across longer distances

pytorch-geometric: MessagePassing

Pytorch geometric's MessagePassing class implements message passing as follows:

1. `message()` implements the MLP ϕ that is to say, it constructs a message $u_j \rightarrow u_i$ for each edge in the edge index
2. `update()` implements ϕ ; it concatenates all inputs before passing them through the MLP
3. `aggregate()` implements the aggregation logic over a neighborhood, i.e. the $\bigoplus_{j \in \mathcal{N}_i}$ operator using a GPU-accelerated *scatter* operation
4. `propagate()` is the initial call to start propagating a message through the graph



Further references

(Veličković, 2023) <https://arxiv.org/pdf/2301.08210.pdf>

(Keisler, 2022) <https://arxiv.org/abs/2202.07575>

(Lam et al., 2023) <https://arxiv.org/abs/2212.12794>

(Sanchez-Lengeling et al., 2021) <https://distill.pub/2021/gnn-intro/>

(Veličković, 2023) <https://geometricdeeplearning.com/lectures/>

(Battaglia et al., 2018) <https://arxiv.org/abs/1806.01261>

(Sanchez-Gonzalez et al., 2020) <https://arxiv.org/abs/2002.09405>